1 - Introduction to Java

Java is an **object oriented language** that models the real world, with greater flexibility, modularity, and reusability. It allows inheritance which is the reason programmers switched from procedural languages to object oriented languages.

Method is a subprogram that is part of a class and can be called in other parts of the program
Procedural-type Methods → Perform an action with a void return (do not return a value)
Functional-type Methods → Calculates and returns a value

Method Header

public int signatureName(int variable) throws IOException

1. modifiers (public, static) [optional]
2. return type (functional methods only)
3. signature (method name, parameters with type)
4. throws clause (throws IOException) [optional]

Advantages of Methods

- Cleaner & more organized code
- Easier to maintain & debug
- More efficient to reuse code
- Better for distributing work
- More secured & protected data

Arguments (Actual Parameters) are values/variables passed into a method

Pass-by-value	Pass-by-reference
a copy of the value of each argument passed to the method, changes will only be modified inside the method while the actual argument remains unchanged	occurs when an object like arrays are passed to a method with its reference location, edits made in the method will affect the values of the original object

Software Design Process

1. Problem Definition - Define the problem in terms of Input, Processing and Output
2. List of Identifiers (Analysis) - Define what an identifier represents and how it is used
3. Algorithm (Design) - A visual representation of a problem solution (e.g Flowcharts)
4. Coding (Implementation) - Writing the program
5. Testing and Debugging - Check for syntax errors and logic errors
6. Maintenance - Update the program

Syntax Error - Error in programming language usage (Missing Parentheses, Brackets, or Braces)
Logic Error - Error in perceiving the problem (Incorrect Algorithm Design)
Stack Overflow Error - Program does not terminate, memory demand exceeds RAM (run-time error)
Time Overhead - Useless work that doesn't directly contribute to the task's main purpose
Memory Overhead - Additional memory required by a data structure beyond the actual data it stores

Code Conventions improve the readability and make programs easier to follow, allowing engineers to understand new code more quickly and thoroughly.

Documentation Comments (/**...*/)	Implementation Comments (//)
- Program Header (name, author, date, description) - Variable Purposes (name, type, description) - Methods Header (description, return, parameters)	- commenting out code - comments about the particular implementation

```
long startTime = System.nanoTime();
   .
long endTime = System.nanoTime();
long totalTime = endTime - startTime; // Measured in nanoseconds
```

public String string = “Hello”, variable = “World”;

Functions	Description	Example
.charAt(x)	Returns the character at position x in a string	string.chartAt(0) *returns H (first letter of string)
.codePointAt(x)	Returns the Unicode of the character at position x of string	string.codePointAt(0) *returns 72 (unicode of H)
.codePointCount(x)	Returns the number of Unicode values found in a string	string.codePointCount(0, 5) *returns 5 (from start index 0 to end index 5)
.equals(x)	Compares a string with another x and returns true if same	string.equals(string) *returns true
.compareTo(x)	(+) = x comes before alpha order (-) = x comes after alpha order	string.compareTo(variable) *returns -15 (“World” comes after “Hello”)
.concat(x)	Connects a string with x	string.concat(variable) *returns HelloWorld
.join(n,x,y)	Joins x and y together with n being the separator	string.join(“ “, string, variable) *returns Hello World
.replace(x, y)	Replace character x in a string with character y	string.replace(‘l’, ‘p’) *returns Heppo
.contains(x)	Checks if a string contain x	string.contains(“lo”) *returns true
.startsWith(x)	Checks if a string starts with x	string.startsWith(“lo”) *returns false
.indexOf(x)	Finds the position of the first occurrence of x in a string	string.indexOf(“l”) *returns 2
.length()	Returns the length of a string	string.length() *returns 5
.toCharArray()	Converts a string to character array	char array = string.toCharArray() *returns array = {H, e, l, l, o}
.toLowerCase	Converts a string to all lower cases	string.toLowerCase() *returns hello
.toUpperCase	Converts a string to all upper cases	string.toUpperCase() *returns HELLO

More about Java: https://www.w3schools.com/java/default.asp

2 - Object-Oriented Language

Class is a template for creating objects, defining attributes (state) and methods (behaviour)
Object (Instance) is an instance of a class that occupies memory and has a unique state
Instance Variable - each object has its own copy created with new and unique memory
Class Variable - declared with "static" and created only once, sharing ONE memory location
State → variable/attribute (what the object has) | **Behaviour** → method (what the object does)

Encapsulation (data hiding) means keeping a variable private and accessing it using methods…
- Protection of data from accidental corruption
- Flexibility and extensibility of the code and reduction in complexity

Access Modifiers control who can access classes, variables, or methods

public	private	protected	default
everywhere	same class only *classes cannot be private	same package/subclass	same package

Accessor Method (Getter) is used to determine the value of a variable
Modifier Method (Setter) is used to change the value of a variable
Helper Method is called inside another method within the same class
Constructor Method is called automatically when an object of a class is created, initializing the new object's instance variables (fields) with initial values

Inheritance allows a class to reuse another class's code with the keyword "extends"
Subclass (child) → inherits from another class | **Superclass (base/parent)** → class inherited from
Method Overloading → same method name, different parameters
Method Overriding → subclass provides its own version of a superclass method

Abstract Classes is a class that can't be instantiated (no object) but can be parent of other classes
Abstract Methods is a method without a body, must be implemented by subclasses
Polymorphism - "one method, many forms", achieved with abstract class/method & inheritance

UML (Unified Modeling Language) Diagram is a system used to design classes

```
public class Car { // Class (Template)
    // Attributes (State)
    private String brand; // Private variable
    private String model; // Private variable

    // No-arg constructor method
    public Car() {
       brand = "Unknown";
       model = "Unknown";
    }

    // Parameterized constructor
    public Car(String brand, String model) {
       this.brand = brand;
       this.model = model;
    }
```

```
    // Methods (Behavior) - Encapsulation
    public String getBrand() { // Accessor Method (Getter)
        return brand;
    }
    public void setBrand(String brand) { // Modifier Method (Setter)
        this.brand = brand;
    }
    public String getModel() { // Accessor Method (Getter)
        return model;
    }
    public void setModel(String model) { // Modifier Method (Setter)
        this.model = model;
    }
}

public class Car_Super extends Car {
    private String name; // Private variable
    // No-arg constructor method
    public Car_Super() {
        super(); // Calls constructor of base class
        name = "Unknown";
    }
    // Parameterized constructor
    public Car_Super(String brand, String model) {
        super(brand, model); // Calls constructor of base class
        name = "Unknown";
    }

    public void drive() {
        System.out.println(getName() + " " + getBrand() + " " + getModel());
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}

public class Main { // Class (Create & Use objects)
    public static void main(String[] args) {
        // Creating objects (instances) of the Car class
        Car_Super myCar = new Car_Super(); // Object 1
        Car_Super sisterCar = new Car_Super("Porsche", "Taycan"); // Object 2
        Car_Super momCar = new Car_Super(); // Object 3
        momCar.setBrand("BMW");
        momCar.setModel("M3");
        momCar.setName("JB");
        myCar.drive();
        sisterCar.drive();
        momCar.drive();
    }
}
// Output: Unknown Unknown Unknown, Unknown Porsche Taycan, JB BMW M3
```

3 - Data Types

Name (identifier) is given to identify specific memory location instead of numeric location

Primitive data types start with a lowercase letter (e.g. int), while **non-primitive data types** typically start with an uppercase letter (e.g. String). Primitive data types always hold a value, whereas non-primitive types can be null. Numerical data inputs are accepted as a string type

Scope of Variable is the part of the program where the variable can be accessed or referenced. Scope refers to the accessibility/visibility of a variable and can be declared in several places:

- Parameters of methods (in method signature) [global]
- Method body (local variable) [local]
- Statement block (like a for loop or a while loop) [block]

Type	Size (bits)	Default	Type	Size (bits)	Default
boolean	1	False	int	32	0
char	16	'\u0000'	long	64	0
byte	8	0	float	32	0.0
short	16	0	double	64	0.0

Variables	capitalize every word except the first (e.g. int numberOfMarks;)	Constants	capitalize every letter of the word e.g. final double NUMBER = 3.14;
Methods	capitalize every word except the first (e.g. public void main())	Classes	capitalize each word e.g. public class HelloWorld

Constant is a variable that cannot be changed once assigned, making revising a program easier with less chance for error.

```
final type NAME = number;
```

e.g.
```
final double NUMBER = 3.141592654;
```

Casting allows converting variables from one data type to another. However, t is not possible to cast a boolean type to any other type or vice versa.

```
int i = 5;
short d = (short) i;
```

e.g.
```
int average;
average = (int) (3.14 + 2.71)/2;
```

Some units may be lost during narrowing conversion. For example, number 259 (100000011) must be squeezed into a single byte. Since byte only accepts the first 8 digits, the first digit (256) has been discarded, with only 3 (00000011) remaining.

Arithmetic Operators

Operator	Symbol	Description	Example
Addition	+	Adds together two values	14 + 6
Subtraction	-	Subtracts one value from another	20 - 6
Multiplication	*	Multiplies two values	14 * 2
Division	/	Divides one value by another	28 / 4
Modulus	%	Returns the division remainder	7 % 3 = 1
Increment	++	Increases the value of a variable by 1	++x
Decrement	−−	Decreases the value of a variable by 1	−−x

Assignment Operators

Expression	Meaning	Expression	Meaning
x += y	x = x + y	x *= y	x = x * y
x -= y	x = x - y	x /= y	x = x / y

Comparison Operators

Operator	Symbol	Example	Operator	Symbol	Example
Equal to	==	x == y	Greater than	>	x > y
Not equal	!=	x != y	Less than	<	x < y
Greater than or equal to	>=	x >= y	Less than or equal to	<=	x <= y

Logical Operators

Operator	Symbol	Description	Example
And	&&	Returns true if both statements are true	x < 5 && x < 10
Or	||	Returns true if one of the statements is true	x < 5 || x > 10
Not	!	Reverse the result, returns false if true	!(x < 5 && x < 10)

```
int x = 5;
int y = ++x;  // x is 6, y is 6
```

```
int x = 5;
int y = x++;  // x is 6, y is 5
```

4 - Java Class Library

Mathematics

Functions	Description	Example
Math.random()	Generates a random number between 0.0 and 1.0	num = (int) (Math.random*100) + 1 *generate an integer between 1 & 100
Math.abs(x)	Returns the absolute value of the parameter x	Math.abs(-5) *returns +5
Math.round(x)	Rounds x according to usual arithmetical rules	Math.round(3.14) *round to 3
Math.ceil(x)	Returns x rounded up to the nearest integer	Math.ceil(3.14) *round up to 4
Math.floor(x)	Returns x rounded down to the nearest integer	Math.abs(3.14) *round down to 3
Math.max(x, y)	Returns the greatest of values of x and y	Math.max(6, 23) *returns 23
Math.min(x, y)	Returns the smallest of values of x and y	Math.min(6, 23) *returns 6
Math.sqrt(x)	Returns the value of the square root of x	Math.sqrt(5) *returns 25
Math.pow(x, y)	Returns the value of x raised to the power of y	Math.pow(2, 3) *returns 8

Scanner and BufferedReader

BufferedReader

```
import java.io.*;
public class ClassName {
    public static void main(String args[]) throws IOException {
        BufferedReader br = new BufferedReader(new
        InputStreamReader(System.in);
        String num;
        System.out.println ("Please enter a number");
        num = Integer.parseInt(br.readLine);
        System.out.println(num);
    }
}
```

Scanner Class

```
import java.util.Scanner;
public class ClassName {
    public static void main(String[] args) {
        Scanner myScanner = new Scanner(System.in);
        String text;
        System.out.println ("Please enter a word");
        text = myScanner.nextLine();
        System.out.println(text);
    }
}
```

Other Functions: next(), nextInt(), nextDouble(), nextLine(), findInLine(".").charAt(0)

Decimal Format

```
DecimalFormat df = new DecimalFormat("0.00");
final double number = 3.141592653589793;
System.out.println(df.format(number));
// Output: 3.14
```

*java.text.DecimalFormat must be imported

File Handling

```
PrintWriter output = new PrintWriter("TextFile.txt");
File ReadmyFile = new File("TextFile.txt");
Scanner input = new Scanner(ReadmyFile);

output.print("A\nB\nC"); // Outputting into TextFile.txt
output.close();

while(input.hasNext()) {
      System.out.println(input.next()); // Reading from TextFile.txt
}
input.close();
```

*java.io must be imported and throw clause (throws IOException) is required

5 - Loopings & Conditions

For Loop

For loop is used to repeat a specific block of code a known number of times

```
for (int i = 0; i < 10; i++) { // Initialisation, Exit condition, Increment
    .
    .
    .
}
```

While Loop

While loop is used to repeat a specific block of code when a statement is true

```
while (true) { // Only proceeds if statement is true
    .
    .
    .
}
```

Do Loop

Executes at least once before checking the condition

```
do {
    .
    .
    .
} while (true); // Checks the statement after the first execution
```

If Selection

Make choices based on conditions (range of values)

```
if (x > y) { // Proceeds only if x is greater than y
    .
    .
} else if (x < y) { // Proceeds only if y is greater than x
    .
    .
} else { // Proceeds if both cases above are not true
    .
    .
}
```

Case Selection

Make choices based on variable (integers only)

```
switch(variable) {
case 0: // Proceeds if variable equals 0
   .
   .
   break;
case 1; // Proceeds if variable equals 1
   .
   .
   break;
default: // Proceeds if none of the cases above is true
   .
   break;
}
```

Try Catch

Catch input errors (e.g. formatting)

```
while (true) {
      try {
            variable = Integer.parseInt(br.readLine());
           .
           .
            break;
      } catch (NumberFormatException e) { // Invalid input format
            System.out.println(e); // Print the error
      }
}
```

6 - Dimensional Arrays

Arrays are data structures that allow you to hold multiple pieces of data, in a single object

One Dimensional Array

```
double arrayList[] = new double[10]; // Declare an array
double arrayList[] = {0.1, 1.2, 2.3, 3.4, 4.5, 5.6, 6.7, 7.8, 8.9, 9.0};

for (int i = 0; i < arrayList.length; i++)
     System.out.println(arrayList[i]); // Display array values
```

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
arrayList	0.1	1.2	2.3	3.4	4.5	5.6	6.7	7.8	8.9	9.0

Two Dimensional Array

```
double arrayList[][] = new double[3][10]; // Declare a 2D array
double arrayList[][] = {{0.1,1.2,2.3,3.4,4.5,5.6,6.7,7.8,8.9,9.0}, {0.9,9.8,
8.7,7.6,6.5,5.4,4.3,3.2,2.1,1.0}, {1.9,2.8,3.7,4.6,5.5,6.4,7.3,8.2,9.1,0.0}};
arrayList [2] [3] = 4.6; // Declare specific array value


for(int r = 0; r < arrayList.length; r++)
     for(int c = 0; c < arrayList[r].length; c++)
          System.out.println(arrayList[r][c]); // Display array values
```

arrayList	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	0.1	1.2	2.3	3.4	4.5	5.6	6.7	7.8	8.9	9.0
1	0.9	9.8	8.7	7.6	6.5	5.4	4.3	3.2	2.1	1.0
2	1.9	2.8	3.7	4.6	5.5	6.4	.3	8.2	9.1	0.0

Three Dimensional Array

```
int arrayList[][][] = new double[3][10][3]; // Declare a 3D array
arrayList [2][3][4] = 10.5; // Declare specific array value

for(int x = 0; x < arrayList.length; x++) {
     for(int y = 0; y < arrayList[x].length; y++) {
          for(int z = 0; z < arrayList[x][y].length; z++) {
               System.out.println(arrayList[x][y][z]); // Display
          }
     }
}
```

ArrayList is a dynamic data structure that can grow and shrink based on program needs
It's dynamic with length, flexible, and has more methods, but is slower due to memory demands

```
import java.util.*;
ArrayList<String> names = new ArrayList<String>();
names.add("Name1");
names.add("Name3");
names.add(0, "Name0"); // Array is overloaded as list is shifted
names.set(2, "Name2"); // Replaces original value

System.out.println(names.size());
for (int i = 0; i <= names.size(); i++) {
    System.out.println("Index " + i + ": " + names.get(i));
}

// Output:
// 3
// Index 0: Name0
// Index 1: Name1
// Index 2: Name2
```

7 - Recursion & Diagrams

Flow of Execution is the order in which statements are executed
⇒ Methods (Functions/Procedures), Conditions (if/else/…), Iterations (loops) & Recursions

Data Flow Diagrams is a graphical representation of the flow of information for any system
Stack Diagrams represents a method, making it easier to interpret recursions
Stack is a conceptual structure based on the principle of last in first out (LIFO)

Iterative (loop based approach) uses for loops, while loops, and do loops to repeat tasks

Advantages of Iterative	Disadvantages of Iterative
- Easier to understand and come up with - Not much memory or space requirements as it is called only once	- More complex methods (more lines) - It is built using loops that do things over and over again

Recursion is an algorithmic technique where a method calls itself with some part of the task

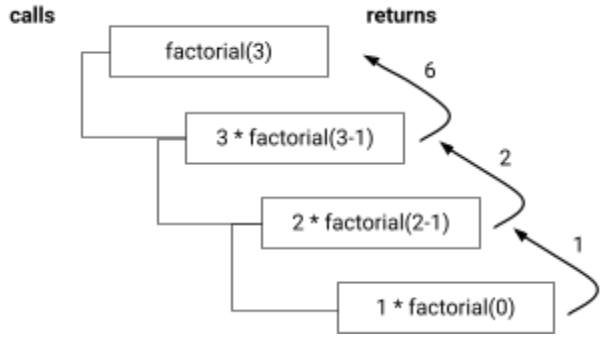
Advantages of Recursion	Disadvantages of Recursion
- Cleaner, shorter, and simpler methods - Program reflects the recursive algorithm used to solve the problem	- Memory Overhead Every method call requires memory space for parameters and local variables - Time Overhead Action of allocating and later releasing memory can lead to time overhead

Three Rules of Recursion

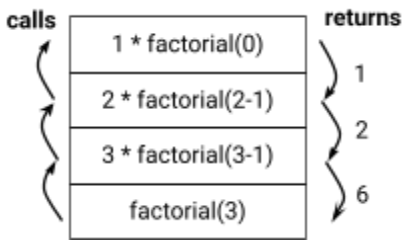
1. Terminating condition (know when to stop)
2. Base case (decide how to take one step)
3. Recursion (break the problem down into steps)

```
private static int factorial(int number) {
      if (number == 0) {
            return 1; // Return 1 when number is 1
      } else {
            return number*factorial(number-1); // Calls itself
      }
}
```

Data Flow Diagram



Stack Diagram



8 - Sorting Algorithms

Sorting is the process of arranging a list of items into a well-defined order

Complexity refers to the resource requirements of a program and can affect performance
Time Complexity (BigO) is the time an algorithm takes to solve a coding problem (efficiency)
Space Complexity is the amount of space an algorithm takes to solve a coding problem

Stability refers to the ability to preserve the relative order of elements with equal values
Adaptiveness is whether it will take the same or less time when the list is partially sorted
Recursive is when an algorithm breaks the problem down and then calling itself
Parallel Sorting divides the process into smaller tasks that are executed simultaneously
In-place Sorting sorts an array by modifying the data directly within its original memory space

Divide and Conquer break problems into small tasks till they are simple enough, then combine

Insertion Sort

take one item at a time and insert it into its correct position like sorting cards

```java
for (int i = 1; i < arr.length; i++) {
    int key = arr[i];
    int j = i - 1;
    while (j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}
```

of Comparisons: $n - 1$ to $\frac{n(n-1)}{2}$
Time Complexity: Best O(n) Worst O(n²)
Space Complexity: O(1)

Stable	Adaptive	Recursive	In-place
✓	✓	✗	✓

Pros	Cons	Applications
- Useful and easy to code - Good for small datasets	- Slow and lots of swaps - Terrible for large datasets	- Small datasets - Nearly sorted data

Selection Sort

repeatedly find the smallest element and swap it into the correct position

```java
for (int i = 0; i < arr.length - 1; i++) {
    int minIndex = i;
    for (int j = i + 1; j < arr.length; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;
        }
    }
    int temp = arr[minIndex];
    arr[minIndex] = arr[i];
    arr[i] = temp;
}
```

of Comparisons: $\frac{n(n-1)}{2}$
Time Complexity: O(n²)
Space Complexity: O(1)

Stable	Adaptive	Recursive	In-place
✗	✗	✗	✓

Pros	Cons	Applications
- Useful and easy to code - Require little memory	- Unstable due to swapping - Non-adaptive and inefficient	- Small datasets - Limited memory

Bubble Sort

swapping adjacent pairs in the list until all adjacent pairs are in order

of Comparisons: $\frac{n(n-1)}{2}$ Time Complexity: O(n²) Space Complexity: O(1)

Stable	Adaptive	Recursive	In-place
✓	✗	✗	✓

Pros	Cons	Applications
- Simple & easy to understand - Require little memory	- Poor time complexity - Slow and inefficient	- Demonstrations - Small and nearly sorted data

```
int temp;
for (int i = 0; i < arrayList.length-1; i++) {
      for (int j = 0; j < (arrayList.length-1-i); j++) {
            if (arrayList[j] > arrayList[j+1]) {
                  temp = arrayList[j];
                  arrayList[j] = arrayList[j+1];
                  arrayList[j+1] = temp;
            }
      }
}
```

Improved Bubble Sort

uses boolean to keep track of swapping to exit the program when sorted

of Comparisons: $n - 1$ to $\frac{n(n-1)}{2}$ Space Complexity: O(1)

Best Case Time Complexity: O(n) Worst Case Time Complexity: O(n²)

Stable	Adaptive	Recursive	In-place
✓	✓	✗	✓

```
boolean swap = true; // Boolean variable to keep track of swaps
int temp, k = 0;
while (swap) {
      swap = false;
      for (int j = 0; j < arrayList.length-k-1; j++) {
            if (arrayList[j] > arrayList[j+1]) {
                  temp = arrayList[j];
                  arrayList[j] = arrayList[j+1];
                  arrayList[j+1] = temp;
                  swap = true;
            }
      }
      k++;
}
```

Quick Sort

picks a pivot, put smaller elements to left and larger to right, recursively sorts the halves

Space Complexity: O(log n)

Pivot is an element chosen from the array that serves as a reference point for partition

Partition is a fundamental step that rearranges a portion of an array around a pivot

Time Complexity	Stable	Adaptive	Recursive	In-place
Best O(n log n) Worst O(n^2)	✗	✗	✓	✓

Pros	Cons	Applications
- Efficient for large datasets - Fast best case performance	- Unstable - Slow worst case performance	- Large, limited datasets - Good average speed

Merge Sort

divide the list into halves, sort each half recursively, then merge them back in order

Space Complexity: O(n)

Time Complexity	Stable	Adaptive	Recursive	In-place
O(n log n)	✓	✗	✓	✗

Pros	Cons	Applications
- Stable and parallel sorting - Consistent time complexity	- Time overhead - Slow on small datasets	- Large datasets (stable sort) - External Sorting

Heap Sort

turn the array into max heap and repeatedly remove the largest element

Space Complexity: O(1)

Heap is the binary tree, data structure

Leaf Node is a node without a child node

Root is the top most node, largest in max-heap, smallest in min-heap

Heapify is the process of converting a binary tree into a max-heap/min-heap recursively

Time Complexity	Stable	Adaptive	Recursive	In-place
O(n log n)	✗	✗	✓	✓

Pros	Cons	Applications
- Consistent time complexity - Require little memory	- Complex to code - Unstable	- Large datasets where stability is not important

9 - Searching Algorithms

Sequential Search goes through the whole list, searching each item until the target is found.

Number of Comparisons: 1 to n (Average: $\frac{(1+n)}{2}$) Average Time Complexity = O(n)

```
public static int sequentialSearch(int[] list, int value) {
	for (int i = 0; i < list.length; i++) {
		if (list[i] == value) {
			return i;
		}
	}
	return -1;
}
```

Binary Search begins by testing the target element against the middle item. If the item is less than the middle item, elements after the middle are disregarded and high = mid - 1. If the item is greater than the middle item, elements before the middle are disregarded and low = mid + 1.

Number of Comparisons: 1 to log n (Average: $\frac{(1+log\ n)}{2}$) Time Complexity = O(log n)

Iterative Approach

```
private static int search(String[] names, String target, int low, int high){
	int mid;
	while(high >= low) {
		mid = (low+high)/2;
		if (target.compareToIgnoreCase(names[mid]) < 0) {
			high = mid - 1;
		} else if (target.compareToIgnoreCase(names[mid]) > 0) {
			low = mid + 1;
		} else {
			return mid;
		}
	}
	return -1;
}
```

Recursive Approach

```
private static int search(String[] names, String target, int low, int high){
	int mid = (low+high)/2;
	if (high >= low) {
		if (target.compareToIgnoreCase(names[mid]) < 0) {
			return search(names, target, low, mid-1);
		} else if (target.compareToIgnoreCase(names[mid]) > 0) {
			return search(names, target, mid+1, high);
		} else {
			return mid;
		}
	}
	return -1;
}
```